

Think Like the R Runtime

A first-principles course in how R actually works, for the statistical and ML community

Mattis Voss

2026-06-18

Table of contents

1. Think Like the R Runtime	1
Preface	3
I. Part I: The Machine	7
2. What the Machine Is Actually Doing	9
II. Part II: Names and Values	17
3. Where Variables Live	19
4. Closures	25
5. Lazy Evaluation and Promises	31
6. The Call Stack	37
III. Part III: Organising Behaviour	41
7. Object Systems: S3, S4, R6	43
8. Reactivity	47
IV. Part IV: Making It Fast	53
9. Performance	55
V. Part V: Code as Data	61
10. Metaprogramming: Code as Data	63
11. The Metacircular Evaluator	69
12. Capstone: One System, Every Idea	77

1. Think Like the R Runtime

Preface

Most R tutorials teach you to *use* functions. This book teaches you to think about what R does when it runs them, all the way down to the numbered boxes of memory the machine is moving numbers between.

If you write any programming language for long enough, you will hit moments when it seems to behave strangely. A function in R captures a variable you thought was local. A reactive expression in a Shiny app fires more often than you expected. A target in a pipeline re-runs when nothing in your data has changed. These are not edge cases. They are the language showing you its underlying machinery, and once you can see that machinery, the strangeness resolves into something you can predict.

This book is a course in how that machinery works, built up in five parts.

Part I starts below the language entirely. Before we can say what an R variable *is*, we need to know what a variable is on the physical machine: what memory actually is, what an address is, what a pointer is, and what the processor is doing billions of times per second. This is the foundation that every later idea rests on. If you have a computer science background, you can skim it. If you do not, it is the most important part of the book, because every apparently magical behaviour in R turns out to be a plain consequence of the ideas in this one chapter.

Part II builds R's model of names and values on top of that foundation: environments, closures, lazy evaluation, and the call stack. By the end you will understand why **targets** can plan a pipeline before running it, and why a closure can remember a value long after the function that created it has returned.

Part III is about organising behaviour: R's object systems (S3, S4, R6) and the reactive programming model that powers Shiny.

Part IV is about speed: why some R code is thousands of times faster than other code that computes the same answer, and how the three layers of caching in the R ecosystem are really one idea applied at three timescales.

Part V is about the deepest idea in the language: that code is itself data, something a program can inspect, build, and run. It starts with metaprogramming in R generally, the toolkit behind formulas and the tidyverse, and culminates in writing a small working interpreter for R, in R. Building a runtime is the surest way to understand one, and it is where every earlier idea in the book comes together at once.

A short **capstone** then shows every one of these ideas living together inside a single realistic system.

1.1. Who this book is for

This book is written for the statistical and machine learning community: people who reach for R to fit models, build reproducible pipelines, and ship interactive apps, and who would like to understand the tool rather than memorise incantations.

Almost no prior R knowledge is assumed. You should be comfortable with the idea of variables, functions, and lists, and you should be numerate, meaning comfortable with abstraction and willing to follow a careful argument. No background in computer architecture is required: Part I builds it from nothing. No JavaScript knowledge is required either, despite the origin of this book's structure, explained below.

1.2. How to read it

Run the code as you go. Each chapter contains short R programs, and almost every one of them is something you can paste into an R session and try. The code blocks are written to be read, not executed automatically, so that the book builds and renders without needing a particular set of packages installed. Where a program produces output, the expected output is shown as comments. Treat any surprise as an invitation to investigate, which is the entire spirit of the book.

Each chapter ends with a short set of exercises and a summary of what you can now predict. The goal is not to finish the book quickly. The goal is to finish a chapter and feel that you can see what was previously invisible.

A note on the code

The R programs in this book are written carefully but are meant to be run and explored by you rather than taken as verified test output. If a result surprises you, run it, read the relevant help page, and reason about it using the model this book gives you. That habit, not the memorisation of any individual result, is what the book is trying to build.

1.3. Acknowledgements and lineage

The structure of this book, its selection of topics and the order in which they build, follows Sumit Saha's course *Think Like the JavaScript Engine*, published on freeCodeCamp in 2026. That course makes a simple and correct argument: you cannot debug, design, or reason about a language by memorising rules, you have to be able to predict what the runtime will do. This book carries that argument over to R.

The mapping between the two languages is faithful. Where JavaScript has scope, R has lexical environments. Where JavaScript has hoisting, R has lazy evaluation. Where JavaScript has prototypes, R has S3, S4, and R6. Where JavaScript has the event loop, R has the reactive graph. In most cases R's version of each idea turns out to be the more interesting one, because R was designed by statisticians who wanted a language they could reason about.

None of the text here is taken from that course; only the pedagogical skeleton is shared, with gratitude.

Part V, on code as data, draws its inspiration from a second source: Abelson and Sussman's *Structure and Interpretation of Computer Programs* (MIT Press), and in particular its idea of teaching a language by building an evaluator for it. The interpreter developed in Chapter 10, and all of the code and prose throughout this book, are original and written for R; what is borrowed from that classic is the pedagogical idea, not its expression. Readers who enjoy Part V should treat *SICP* as the natural place to go deeper.

The two books that go deeper than this course, and that every serious R programmer should eventually read, are Hadley Wickham's *Advanced R* and Norman Matloff's *The Art of R Programming*. Where this book ends, those begin.

1.4. License

This book is free and open source. The prose is licensed under [Creative Commons Attribution 4.0 International](#) (CC BY 4.0), and the code examples under the [MIT License](#). You are free to read it, share it, teach from it, translate it, and build on it, including commercially, as long as you give appropriate credit. See the `LICENSE` and `LICENSE-CODE` files for the full terms.

The one idea

If you take only one thing from this book, take this: there is no magic. Every behaviour described in these pages is a consequence of a program, written in C, managing numbered boxes of memory on your behalf. Once you believe that literally, you can derive the rest.

Part I.

Part I: The Machine

2. What the Machine Is Actually Doing

Before we can talk about how R organises your variables, we have to talk about what a variable physically *is*. Not as a metaphor, but as an arrangement of electrical switches in a chip. This chapter builds, from the level of a single switch up to the R interpreter, the substrate that every later idea in this book rests on.

The payoff is concrete. By the end of this chapter you will be able to answer, from first principles, questions that otherwise have to be memorised:

- Why does assigning `y <- x` sometimes copy a million numbers and sometimes copy nothing at all?
- Why do R environments behave differently from every other object in the language?
- Why is a loop that grows a vector thousands of times slower than one that does not, even though both compute the same answer?
- How can a function “remember” a value long after the call that created it has finished?

Every one of these has the same kind of answer, and that answer lives here.

2.1. Everything is switches

At the very bottom, a computer is an enormous collection of switches, each of which is either on or off. We write “on” as 1 and “off” as 0. One such switch is a **bit**.

A single bit is not much. It can record one yes-or-no fact: true or false, present or absent. But bits combine. Put eight of them in a row and you get a **byte**, which can take any of $2^8 = 256$ distinct patterns, from 00000000 to 11111111. If we agree to read those patterns as numbers, a byte can hold any whole number from 0 to 255. Now we can count.

Everything else is built by agreement on top of this. A number larger than 255 does not fit in one byte, so we use several together: four bytes give 2^{32} , about 4.3 billion patterns, enough for most counting. Text is bytes too. By a near-universal agreement called ASCII, the letter **A** is the number 65, which is the byte 01000001. A decimal value like 3.14 is stored using another agreement (a format called IEEE 754) spread across eight bytes.

Here is where intuition usually goes wrong: **a byte does not know what it means**. The pattern 01000001 is the number 65, and the letter **A**, and part of some larger number, all at once. Meaning comes entirely from how a program chooses to interpret the pattern. The bytes are mute. The program supplies the interpretation.

i Analogy: a wall of light switches

Picture a wall covered in millions of light switches. At any instant the wall is in some configuration of on and off. That configuration *is* the computer's current state. Computing is nothing more than rules for flipping switches based on the current pattern of switches. Everything that follows, every number, every model, every Shiny app, is a very elaborate story we tell about which switches are on.

2.2. Memory is a street of numbered boxes

Now the single most useful mental model in this book.

Imagine a very long street. Along one side, in order, stands a row of identical small boxes. Each box holds exactly one byte. Each box has a number painted on its front: the first box is box 0, the next is box 1, then box 2, and so on, up into the billions. That painted number is the box's **address**.

This street is your computer's memory, its RAM. That is, almost literally, all that RAM is: a vast, ordered array of byte-sized boxes, each one individually reachable by its address.

When a program wants to store a value, it does two things. It picks one or more boxes, and it writes the bytes of the value into them. To retrieve the value later, it has to know *which* boxes, which means it has to know the address.

Here is the first deep idea of the book.

! Key idea: a variable is a name in a side-table

The machine has no notion of the name `x`. What physically exists is a value sitting in some boxes, plus an address that tells you where those boxes are. A “variable” is a human convenience: a label that the programming language keeps in a separate lookup table, mapping the name `x` to the address where `x`'s value lives.

Hold onto that, because in the next chapter we will discover that R's **environments** are exactly such lookup tables, and that single fact is the whole secret behind how scope works.

Let us make it concrete. Suppose somewhere inside a running R session the following happens.

```
x <- 10
```

Underneath, the runtime does roughly this. It finds some free boxes on the street, say starting at address 4096, and writes the bytes representing the number 10 into them. Then, in a side-table for the current scope, it records an entry: the name "`x`" maps to address 4096. When you later write `x + 5`, the runtime looks up "`x`" in the side-table, gets the address 4096, reads the bytes there, interprets them as the number 10, and adds 5.

You never see the address. The whole point of a programming language is to let you say `x` and not 4096. But the address is what is really there, and several of R's most confusing behaviours become obvious the moment you remember that names and addresses are two different things.

2.3. A pointer is a value that is an address

Since an address is just a number, and a box can hold a number, a box can hold an *address*. A box whose contents are the address of another box is called a **pointer**. We say the first box “points to” the second.

i Analogy: a sticky note with a locker number

A pointer is like a sticky note that reads “the thing you want is in locker 4,815”. The note is small and cheap to carry around and copy. The contents of the locker might be huge. If I hand you a copy of the note, we are now both looking at the same locker. Neither of us has copied what is inside it. We have only copied the note.

This sounds almost too simple to be important. It is, in fact, one of the most powerful ideas in all of computing, and it is the key to three things that confuse nearly every R programmer:

1. Why `y <- x` sometimes copies data and sometimes does not. (Chapter 8: it copies the note, not the locker, until someone needs to change the contents.)
2. Why environments behave unlike every other R object. (Chapter 2: an environment is handled through a pointer that is never silently copied.)
3. How a single R list can hold a number, a data frame, and a function all at once. (The list does not contain those things; it contains pointers to them, and a pointer is the same small size no matter how large the thing it points to.)

Picture two names, `x` and `y`, after `y <- x`. There is one value on the heap, and two pointers aimed at it.

name table	memory (the heap)
-----	-----
x -----\	
	>-----> [the vector 1, 2, 3, ...]
y -----/	

Copying `y <- x` did not duplicate the vector. It duplicated the arrow. This picture, and what must happen to it when someone tries to change the vector through `y`, is the entire content of “copy-on-modify”, which we reach in Chapter 8.

2.4. Two neighbourhoods: the stack and the heap

Programs do not treat memory as one undifferentiated street. They carve it into regions with different rules. Two of those regions matter for everything that follows.

The **stack** is the region used for the short-lived bookkeeping of function calls. It behaves exactly like a stack of plates: you may add a plate to the top, or remove the plate from the top, and nothing else. When a function is called, the program pushes a fresh plate, called a **stack frame**, onto the top. That frame holds the call’s local variables and, importantly, the address to jump back to when the function finishes. When the function returns, its frame is popped off the top

2. What the Machine Is Actually Doing

and instantly forgotten. The stack is fast and fully automatic, but its discipline is strict: a function’s frame vanishes the moment the function returns.

The **heap** is the region for values that need to outlive the function call that created them, or that are too large, or whose size is not known in advance. Getting a box from the heap takes more work than pushing onto the stack, and something has to decide when a heap box is no longer needed and may be reused. In low-level languages the programmer makes that decision by hand. In R, a part of the runtime called the **garbage collector** does it automatically, reclaiming any heap memory that nothing points to any more.

This split pays off twice later in the book, so it is worth fixing now.

! Key idea: the stack is temporary, the heap persists

Local bookkeeping for a function call lives on the stack and dies when the call returns. The actual data your program works with lives on the heap and survives as long as something points to it. Closures (Chapter 3) work precisely because R can keep an environment alive on the heap after the stack frame that created it is long gone. The “call stack” you inspect after an error (Chapter 5) is, quite literally, this stack of frames.

i Analogy: scratch paper and a filing cabinet

The stack is a pad of scratch paper. For each task you tear off a fresh sheet, jot down what you need while working, and throw the sheet away the instant the task is done. The heap is a filing cabinet. You put things there when they need to outlast the current task, and a clerk (the garbage collector) periodically removes any file that no sticky note still refers to.

2.5. The processor: a fast, literal clerk

So far we have memory, but nothing that acts. The actor is the **CPU**, the central processing unit.

The CPU does astonishingly little, astonishingly fast. In a tight loop, billions of times per second, it does three things: it **fetches** the next instruction from memory, **decodes** what that instruction means, and **executes** it. Then it does it again.

The instructions it understands are tiny and utterly concrete. There is no instruction for “fit a regression”. The instructions are more like “load the bytes at address A into register 3”, “add register 3 and register 4 and put the result in register 5”, and “if register 5 is zero, jump to the instruction at address B”. Everything your computer does, every model fit and every web page, is a tower of abstractions resting on billions of these minuscule steps.

Registers are the handful, a few dozen, of tiny storage slots physically inside the CPU. They are the only places the CPU can actually do arithmetic. To add two numbers that live in memory, the CPU must first load each into a register, add the registers, and store the result back out to memory. Registers are blindingly fast precisely because there are so few of them and they sit right on the chip.

i Analogy: a clerk with a tiny desk

Imagine an immensely fast clerk working at a tiny desk in a colossal warehouse (memory). The desk has room for only a few items at a time (the registers). To combine two items stored deep in the warehouse, the clerk must walk out, fetch each one to the desk, do the work at the desk, and carry the result back to a shelf. The clerk is so fast that the walking, not the thinking, is what takes the time. Keep that image: in modern computers, moving data is usually the expensive part, not computing on it.

2.6. The memory hierarchy, and why nearness is speed

That last analogy points at the most important performance fact in modern computing. There is a cruel trade-off in hardware: memory that is fast is also small and expensive, while memory that is large is slow and cheap. No single kind of memory is both big and fast. So computers stack several kinds into a **hierarchy**, fastest and smallest at the top:

- **Registers:** a few dozen slots, inside the CPU, access in well under a nanosecond.
- **Cache** (levels L1, L2, L3): kilobytes to a few megabytes, very near the CPU, a few nanoseconds.
- **Main memory** (RAM): gigabytes, around 100 nanoseconds, which is roughly a hundred times slower than cache.
- **Disk or SSD:** terabytes, microseconds to milliseconds, dramatically slower again.

When the CPU needs a byte, it first checks the nearest cache. If the byte is there (a cache **hit**), wonderful. If not (a cache **miss**), it must go to a slower level, which can be a hundred times the wait. And here is the detail that drives a great deal of real-world performance: when the CPU fetches one byte from RAM, it does not fetch just that byte. It grabs a whole neighbouring chunk, called a **cache line**, typically 64 bytes, on the bet that you will very soon want the bytes sitting right next door.

That bet is the physical reason that walking through data laid out *contiguously* in memory is far faster than chasing pointers scattered across the heap. With contiguous data, every expensive trip to RAM brings back 64 useful bytes at once. With scattered pointers, each trip may bring back 63 bytes you do not want. We bank this insight now and cash it in Chapter 8, where it explains why R’s vectorised operations are fast and its pointer-chasing patterns are slow.

! Key idea: locality is speed

Data that sits together is processed together, quickly. Data that is scattered forces the processor to wait on memory again and again. A surprising amount of “make it faster” in numerical computing is really “make the data contiguous so the cache bet pays off”.

2.7. What R adds: an interpreter is just a program

We now have enough to say precisely what R *is*.

2. What the Machine Is Actually Doing

R is a program, written mostly in the C language, that runs on the machine we have just described. When you type R code, you are not speaking to the CPU directly. You are handing text to the **R interpreter**, a C program that reads your code and, on your behalf, performs the corresponding manipulations of memory: allocating boxes on the heap, writing values into them, recording names and addresses in side-tables, and orchestrating the CPU to do the arithmetic.

Every R value you create is, underneath, a C structure living on the heap. R's internals call these structures **SEXP**s (short for S-expressions). A SEXP carries a small header (what type is this: an integer vector? a list? a function? and some bookkeeping, including a count of how many things point to it, which we will need in Chapter 8) followed by either the data itself or a pointer to it.

An **R environment**, the protagonist of the next chapter, is one particular kind of SEXP. In essence it is a hash table mapping names to the SEXPs they are bound to, together with a pointer to a parent environment. Read that sentence again, because once you believe it literally, the entire behaviour of scope, closures, and reference semantics stops being a list of rules and becomes a set of consequences you can derive on your own.

! Key idea: there is no magic

Everything in the rest of this book is a consequence of a C program managing numbered boxes of memory for you. When R surprises you, the explanation is never magic. It is always a fact about names, addresses, pointers, the stack, the heap, or the cache, and you now know all six.

2.8. What you can now predict

You have not written any “real” R yet, but you can already reason about it. With only the ideas in this chapter, you should be able to explain, at least in outline:

- Why two variables can refer to the same underlying data, and what copying really copies.
- Why some things a function creates disappear when it returns, while others survive.
- Why the *layout* of your data in memory can matter as much as the algorithm.
- Why an interpreted language like R can be slow in a loop yet fast on a whole vector at once.

Each of these becomes a precise, mechanical story in the chapters ahead. We begin with the side-table that maps names to values: the environment.

2.9. Exercises

1. In your own words, explain the difference between a name, an address, and a value. Use the locker analogy, then restate it without any analogy at all.
2. Suppose **a** is a vector of one million numbers and you write **b <- a**. Before reading Chapter 8, predict what is copied at that moment: the million numbers, or something much smaller? Justify your answer using only the pointer picture from this chapter.

3. A function creates a local counter, increments it, and returns. Using the stack-and-heap distinction, explain why that counter would normally be gone after the function returns, and sketch what would have to be true for it to survive. (You are predicting the mechanism behind closures, which is Chapter 3.)
4. Explain, using the cache-line idea, why summing the elements of one contiguous vector of a million numbers can be much faster than summing a million numbers each stored in a separate object linked by pointers, even though the count of additions is identical.

Part II.

Part II: Names and Values

3. Where Variables Live

In the foundations chapter (Chapter 1) we established that a variable is a name in a side-table, mapping a string to an address on the heap. R has a specific name for that side-table. It calls it an **environment**, and it makes environments into real, inspectable objects. This chapter is about what they are and how lookups walk between them. Almost everything later in the book is a consequence of this one mechanism.

3.1. An environment is a side-table with a parent

Recall the picture from the foundations chapter: when you write `x <- 10`, the runtime puts the value 10 in some boxes on the heap and records, in a side-table, that the name "x" maps to that location. An environment *is* that side-table. Concretely it is a hash table from names to values (technically to the SEXPs those names are bound to), plus one extra pointer: a link to a **parent** environment.

That parent pointer is the entire trick. When R needs the value of a name and does not find it in the current environment, it follows the parent pointer and looks in the parent, then the parent's parent, and so on. The search stops when the name is found, or when R reaches the topmost environment (the empty environment) and gives up with an error.

```
x <- 10

environment()      # the environment you are typing into,
                   # known as the global environment
ls(environment())  # list every name bound in it
get("x", envir = environment()) # retrieve the value bound to "x"
```

So an environment is two things bolted together: a table of name-to-value bindings, and a pointer to where to look next. Keep that shape in mind. We will see it reused, almost unchanged, as the basis of closures, of R6 objects, and of Shiny's reactive bookkeeping.

3.2. Why one environment is not enough

A useful R session quickly contains many environments, not one. The reason is functions, and it follows directly from the stack-and-heap distinction in the foundations chapter (Chapter 1).

Consider defining and calling a function.

3. Where Variables Live

```
f <- function(x) {  
  y <- x * 2  
  y + 1  
}  
  
f(3)      # 7
```

When R runs `f(3)`, it needs somewhere to keep the local names `x` (bound to 3) and `y` (bound to 6). Those names must not collide with any `x` or `y` you happen to have in the console. So R creates a fresh environment for this one call, the call's stack frame from the foundations chapter (Chapter 1), and binds the local names there. The environment lives for the duration of the call and is normally discarded when `f` returns.

But functions also mention names that are not their own arguments.

```
scaling_factor <- 0.5  
  
scale <- function(x) x * scaling_factor  
  
scale(10)      # 5
```

When `scale(10)` runs, R cannot find `scaling_factor` in the function's own call environment. So it follows the parent pointer and looks in a wider environment. This is the second idea the chapter turns on: every environment has a parent, and name lookup walks the chain of parents until the name is found or the chain runs out.

3.3. The search path

Here is the chain that exists during a typical session. Each entry is an environment, and the parent pointers run down the list.

```
search()  
# [1] ".GlobalEnv"      "package:stats"  
# [3] "package:graphics" "package:grDevices"  
# [5] "package:utils"    "package:datasets"  
# [7] "package:methods"  "Autoloads"  
# [9] "package:base"
```

The chain runs from the global environment, where you type, down to the base environment, where built-in functions like `sum` and `length` live. When you reference a name, R walks this chain in order. This also explains an occasional source of confusion: if two packages both export a function called `select`, the one you loaded *more recently* wins, because `library()` inserts each new package near the front of the chain, ahead of the older one.

i Analogy: a chain of dictionaries

Think of each environment as a dictionary, and the parent pointer as a note at the back saying “if you did not find your word here, try this other dictionary next”. To look up a word you start in your own dictionary and keep following the notes outward until you find a definition or run out of dictionaries.

3.4. Lexical scoping: where a function looks

One subtle question remains, and it is the heart of the chapter. When `scale` needs `scaling_factor`, *whose* parent chain does it walk? There are two candidates. It could walk the chain of the environment where the function was **defined**, or the chain of the environment it was **called from**.

R uses the first. This rule is called **lexical scoping**: every function carries a pointer to the environment in which it was created, and free names (names that are not its arguments) are looked up starting there. The alternative, resolving names from wherever the function happens to be called, is called dynamic scoping, and R rejects it, as do almost all modern languages, because it would make a function’s meaning depend on its caller, and reasoning would collapse.

The cleanest demonstration is a function that captures a value that is later changed.

```
x <- "global"

make_printer <- function() {
  x <- "local to make_printer"
  function() print(x)      # this inner function is
                           # defined inside make_printer
}

p <- make_printer()
x <- "changed in global"

p()
# [1] "local to make_printer"
# Not "changed in global", and not "global".
```

Read that slowly. The inner function does not even exist when `x <- "global"` runs. By the time the inner function is finally called, the line `x <- "changed in global"` has executed. Yet the inner function prints the value of `x` that was in scope *where it was defined*. The function held a pointer to its definition environment, and, exactly as the foundations chapter (Chapter 1) predicted, that pointer kept the environment alive on the heap long after `make_printer` returned and its stack frame was gone. The next chapter, on closures, is entirely about this.

! Key idea: a function is body plus environment

A function's behaviour depends on three things: its arguments, its body, and the environment in which it was defined. Two functions with identical arguments and identical bodies can produce different results because their definition environments differ. This is the single most important fact about R functions, and it is a direct consequence of lexical scoping.

3.5. Inspecting environments

R lets you look at all of this directly. The `rlang` package provides the most readable tools.

```
library(rlang)

f <- function() {
  a <- 1
  b <- 2
  env_print()      # prints this environment and its parent
}
f()
# <environment: 0x...>
# parent: <environment: global>
# bindings:
#   * a: <dbl>
#   * b: <dbl>
```

You can also build environments by hand and use them as plain data structures. This is the trick behind R6 classes, behind the cache in `memoise`, behind Shiny's tracking of reactive dependencies, and behind `targets`' record of pipeline state. We will meet all four.

```
e <- new.env(parent = emptyenv())
e$counter <- 0
e$counter <- e$counter + 1
ls(e)      # "counter"
identical(e, e) # TRUE
```

3.6. Environments are the one exception to copy-on-modify

That last line points at something important, and it reaches straight back to pointers in the foundations chapter (Chapter 1). Almost every object in R is handled with **copy-on-modify** semantics, which we study properly in Chapter 8: if two names point to the same value and you modify it through one of them, R quietly makes a copy first so the other name is unaffected.

Environments are the single deliberate exception. They are passed and assigned by **reference**, meaning the pointer is shared and never silently copied. If you pass an environment to a function and the function modifies it, the change is visible everywhere, because every name involved is pointing at the very same table on the heap.

```
e <- new.env()
e$value <- 1

bump <- function(env) env$value <- env$value + 1

bump(e)
e$value    # 2, the change is visible outside the function
```

This is not a quirk. It is the property that makes environments useful as the storage behind mutable objects. When you want state that several functions can share and change, an environment is the tool, precisely because it is the one thing R will not copy behind your back.

3.7. Why this matters for your work

3.7.1. For a targets pipeline

Every `tar_target(name, command)` wraps `command` as a function, and that function carries a definition environment, usually the environment in which your `_targets.R` file was sourced. If you accidentally leave a large object in that environment (a multi-gigabyte data frame loaded while testing, say), targets that close over it can treat it as part of their dependency footprint and may serialise it to disk. The fix is structural and follows directly from lexical scoping: define your target functions in a separate `R/` directory, sourced into a clean environment, so that the definition environment contains only your helper functions and none of your scratch data.

3.7.2. For a Shiny app

The `server` function runs once per user session. Every `reactive()`, `observe()`, and `output$...` assignment inside it closes over that session's server environment. This is why a plain `x <- 0` at the top of `server` acts as per-session state, private to one user, while an object created *outside server*, in `global.R`, is shared across all sessions: each session's closures walk their parent chain up into the same outer environment and find the one shared copy there. Scope, in other words, is your main tool for deciding what is per-user and what is global.

3.8. What you can now predict

- Why a function can see variables from where it was written, but not from where it is called.
- Why moving a heavy computation outside a Shiny `server` function shares it across users, while leaving it inside isolates it per user.
- Why environments, alone among R objects, let a change made in one place be seen in another.
- Why a pipeline tool can accidentally drag a large object into a target, and how scope discipline prevents it.

3.9. Exercises

1. Without running it, predict what each of the three `cat()` calls below prints. Then run it and explain any surprise using the side-table picture from this chapter.

```
x <- 1
f <- function() {
  cat(x, "\n")
  x <- 2
  cat(x, "\n")
}
f()
cat(x, "\n")
```

2. Use `new.env()`, `assign()`, and `get()` to build a tiny key-value store. Make it reject re-assignment to a key that already exists. You will not need any package.
3. Write `parent_chain(env)`, returning a list of all ancestor environments of `env`, ending at the empty environment. Test it on the global environment, then run `library(dplyr)` and test again. What changed in the chain, and where?
4. Demonstrate the reference behaviour of environments. Create an environment, pass it to two different functions that each modify it, and confirm that both changes are visible afterward. Then do the “same” thing with a list and explain why the list does not behave the same way.

4. Closures

Every function in R is a closure. Most R programmers do not know this, and the ones who do are usually the ones writing the libraries everyone else depends on. This chapter explains what that sentence means, why it follows directly from the previous two chapters, and how it quietly powers function factories, the cache in `memoise`, R6 objects, and Shiny reactives.

4.1. The three parts of a function

We can now state precisely what an R function is. A function is exactly three things: its **arguments** (the names between the parentheses), its **body** (the expression to evaluate), and its **definition environment** (the environment in which it was created, introduced in the previous chapter). R lets you pull out all three.

```
f <- function(x, y = 2) x + y

formals(f)      # the arguments: x, and y with default 2
body(f)         # the body: x + y
environment(f)  # the definition environment
```

When you call `f(3)`, four things happen, and they are exactly the heap-and-stack mechanics from Chapter 1. R creates a new environment whose parent is `environment(f)`. It binds `x = 3` and `y = 2` into that environment. It evaluates the body there. Then it returns the result and discards the environment, *unless something else still points to it*. That italicised clause is the whole definition of a closure.

! Key idea: a closure is a function plus a captured environment

A closure is a function together with a reference to its definition environment that survives the call which created it. Because every R function carries a pointer to its definition environment, every R function is technically a closure. The idea only becomes interesting when that captured environment holds state the function can read or change.

4.2. The factory pattern

The clearest demonstration of closures is a function that returns a function.

4. Closures

```
make_adder <- function(n) {  
  function(x) x + n  
}  
  
add3 <- make_adder(3)  
add5 <- make_adder(5)  
  
add3(10)      # 13  
add5(10)      # 15  
  
environment(add3)$n    # 3  
environment(add5)$n    # 5
```

Trace it through the Chapter 1 picture. We called `make_adder` twice. Each call created a fresh environment on the heap containing a binding for `n`. Each call returned an inner function whose definition environment is that fresh environment. The two returned functions therefore have identical arguments (`x`) and identical bodies (`x + n`), yet they behave differently, because they carry different definition environments. The last two lines prove it by reaching directly into each closure’s captured environment and reading the `n` it kept.

This pattern is everywhere in the R ecosystem once you start looking. The `scales` package gives you `label_dollar(prefix = "$")`, which returns a formatting function. The `purrr` package gives you `partial(f, x = 1)`, which returns a function with one argument pinned. Many statistical constructions are naturally families of functions: a smoothing routine that returns a fitted function, a likelihood builder that returns a function of the parameters. Whenever the right design is “a configurable family of functions”, the closure-based factory is usually the cleanest way to express it.

i Analogy: a cookie cutter with a setting

`make_adder` is a cookie cutter with a dial. You set the dial to 3, stamp out a function, and that function “remembers” the 3 forever because the setting was baked into its own private environment at the moment of stamping. Set the dial to 5 and stamp again, and you get a different function with a different memory. The cutter (the factory) and the cookies (the returned closures) are different things.

4.3. Hidden state and the superassignment arrow

Closures give R something that looks like private, mutable state. The mechanism is the **superassignment** operator, written `<<-`. The ordinary arrow `<-` creates or updates a binding in the *current* environment. The double arrow walks *up* the parent chain looking for an existing binding of that name, and modifies the first one it finds.

```
make_counter <- function(start = 0) {  
  count <- start  
  list(  
    count = count  
  )  
}
```

```

increment = function() {
  count <- count + 1      # modify count in the parent environment
  invisible(count)
},
reset = function() {
  count <- start
  invisible(count)
},
value = function() count
)
}

c1 <- make_counter()
c1$increment(); c1$increment(); c1$increment()
c1$value()      # 3

c2 <- make_counter(100)
c2$value()      # 100, completely independent of c1

```

Each call to `make_counter()` creates a fresh environment containing `count`. The three inner functions all close over *that same* environment, so they share one counter. From outside, `c1` and `c2` look like little objects with methods, and they keep separate state, but there is no class system here at all, only environments and functions. This is sometimes called **functional object-oriented programming**, and it is the conceptual seed from which R6 grows later in the book.

Notice that this only works because of the reference behaviour of environments from the previous chapter. The three inner functions are not each given a private copy of `count`. They share one environment by pointer, so when `increment` changes it, `value` sees the change.

4.4. The classic trap: closures in a loop

The single most common closure bug in R involves capturing a loop variable. The example is small, but the same shape appears in larger Shiny apps and package code, so it is worth burning into memory.

```

fs <- list()
for (i in 1:3) {
  fs[[i]] <- function() i
}

fs[[1]]()  # 3, not 1
fs[[2]]()  # 3, not 2
fs[[3]]()  # 3

```

Why does every stored function return 3? Because all three inner functions close over the *same* environment, the one the `for` loop ran in. There is only one `i` in that environment. By the time

4. Closures

you call the stored functions, the loop has finished and that single `i` holds its final value, 3. The closures faithfully look up `i` and report what they find. Nothing is broken; the functions are doing exactly what lexical scoping says they must.

The fix is to give each function its own environment with its own `i`. The standard way is to use a function that creates a fresh call environment per iteration.

```
fs <- lapply(1:3, function(i) {  
  function() i  
})  
  
fs[[1]]() # 1  
fs[[2]]() # 2  
fs[[3]]() # 3
```

Now each iteration is a separate call to the function passed to `lapply`, and each call makes its own environment with its own `i`. The functions returned from those calls close over different environments and report different values. This is also exactly why `purrr::map` is immune to the bug: it uses the same per-element call mechanism.

4.5. Memoise, in eight lines

To make all of this concrete, here is the entire idea of `memoise::memoise()`, built from scratch using only a closure and an environment.

```
memoise <- function(f) {  
  cache <- new.env(parent = emptyenv())  
  function(...) {  
    key <- rlang::hash(list(...))  
    if (is.null(cache[[key]])) {  
      cache[[key]] <- f(...)  
    }  
    cache[[key]]  
  }  
}  
  
slow <- function(x) { Sys.sleep(1); x^2 }  
fast <- memoise(slow)  
  
system.time(fast(4)) # about 1 second  
system.time(fast(4)) # essentially zero, served from cache
```

Look at the structure against everything so far. We make an empty environment to use as a cache, living on the heap. We return a wrapper function that, on each call, hashes its arguments, looks the hash up in the cache, and either returns the stored value or computes a fresh one and stores it. The wrapper closes over both `f` and `cache`. Each call to `memoise` mints a new, private

cache. The real `memoise` package adds robustness (disk backends, expiry, better hashing) but the engine is exactly this. Caching, at bottom, is just a closure holding a table.

i This is the seed of two later chapters

Hold this eight-line `memoise` in mind. In Chapter 7 we will see that a Shiny `reactive()` is essentially this same pattern, a closure wrapping a computation and caching its result, with the extra twist that the cache is invalidated automatically when inputs change. In Chapter 8 we will see that `targets` is again this pattern, with the cache living on disk and surviving across sessions. One idea, three scales.

4.6. Why this matters for your work

Once the factory pattern is clear, you will start to see it in your own code, and reaching for it will make that code simpler. When you find yourself writing several near-identical functions that differ only in one or two settings, the right design is almost always a single factory that captures the settings in a closure and returns a clean function that never mentions them. This shows up when building a family of loss functions for an optimiser, a set of log-likelihoods over different priors, or a collection of themed plotting or table-formatting helpers for a report. The configuration goes into the closure; the interface stays simple.

4.7. What you can now predict

- Why two functions with the same body can behave differently.
- Why a counter built from a closure keeps its own private, persistent state.
- Why all the functions made in a naive loop end up sharing one value, and how to fix it.
- Why caching, reactivity, and pipeline tools are, underneath, the same closure-plus-table idea.

4.8. Exercises

1. Modify `make_counter()` to accept a `by` argument so the step size is configurable, and add a `history()` method that returns every value the counter has ever held.
2. Write `throttle(f, n)`, returning a wrapped version of `f` that actually calls `f` only on every `n`-th invocation and returns `NULL` the rest of the time. Use only closures, no extra packages.
3. Extend the eight-line `memoise` so that cache entries expire after a fixed wall-clock duration. Where, physically, does the expiry time for each entry live? What goes in the cache environment now, and what stays in the closure?
4. Predict the output of the loop trap example before running it, then fix it two different ways: once with `lapply`, and once by keeping the `for` loop but forcing a fresh binding per iteration with `local()`. Explain, in terms of environments, why each fix works.

5. Lazy Evaluation and Promises

In R, function arguments are not evaluated when you pass them. They are wrapped in a small object, parked, and evaluated later, only if and when they are actually used. This is the single deepest feature separating R from almost every mainstream language, and it is the mechanism that lets `dplyr` understand `filter(df, x > 3)` and lets `targets` plan a pipeline before running a line of it.

5.1. A surprising experiment

Start with a function whose argument announces when it is evaluated.

```
noisy <- function(label) {  
  cat("evaluating", label, "\n")  
  label  
}  
  
f <- function(a, b) {  
  cat("entered f\n")  
  cat("about to use a\n")  
  a  
  cat("about to use b\n")  
  b  
}  
  
f(noisy("A"), noisy("B"))  
# entered f  
# about to use a  
# evaluating A  
# about to use b  
# evaluating B  
# [1] "B"
```

Read the output carefully. `entered f` prints *before* either argument is evaluated, even though `noisy("A")` and `noisy("B")` are written right there in the call. Each argument is evaluated only at the moment its name is first used inside the body. That is lazy evaluation, and once you have seen it you cannot unsee it.

5.2. The mechanism: promises

When R prepares to call a function, it does not evaluate the arguments and then enter the function. Instead, for each argument, it creates a small object called a **promise**. Building on Chapter 1, a promise is a structure on the heap holding three things: the original unevaluated expression, a pointer to the environment where that expression should eventually be evaluated (the calling environment), and an empty slot for the resulting value.

When the body first uses the argument by name, the promise is **forced**: its expression is evaluated in the stored environment, the result is written into the value slot, and from then on the promise behaves like a plain value. Forcing happens at most once. Ask for the argument ten times and the expression runs once; the other nine reads return the cached value.

i Analogy: an IOU

A promise is an IOU. When you call a function, you do not hand it the goods, you hand it a note describing where to get them and the address to fetch them from. The function only redeems the note if it actually needs the goods. If it never looks, the goods are never fetched, which is why an unused argument that would have thrown an error often causes no error at all.

5.3. Consequences you can rely on

5.3.1. Default arguments can refer to other arguments

Because defaults are themselves promises, evaluated inside the function's own environment when first used, a default can depend on another argument.

```
f <- function(x, y = x * 2) x + y

f(3)          # 9, because y defaults to x * 2 = 6
f(3, 10)      # 13, because y is supplied
```

The default `x * 2` is not computed when the function is defined. It is a parked expression that is evaluated, if needed, inside the call, where `x` is in scope.

5.3.2. You can capture the expression, not just the value

R lets you reach into a promise and pull out the unevaluated expression. The built-in primitive is `substitute()`.

```
inspect <- function(x) {
  expr <- substitute(x)          # the unevaluated expression
  cat("Expression:", deparse(expr), "\n")
  cat("Value:      ", eval(expr), "\n")
}
```

```
a <- 5
inspect(a + 7)
# Expression: a + 7
# Value:      12
```

This is the foundation of what R calls **non-standard evaluation**. When `dplyr::filter(df, x > 3)` works even though there is no `x` defined in your console, it works because `filter` grabs the unevaluated expression `x > 3` from its promise, looks inside it, and evaluates it in the data frame’s environment, where `x` is the name of a column, instead of in your console’s environment. The promise carried both the expression and a choice of where to run it, and `filter` overrode the where.

5.4. Quosures: a friendlier interface

The `rlang` package wraps promises in a tidier, first-class object called a **quosure**: an expression bundled with the environment it came from. Promises are an internal mechanism that R uses as it runs. Quosures are objects you can store in a variable, pass to other functions, modify, and evaluate on demand.

```
library(rlang)

capture <- function(x) {
  q <- enquo(x)                                # capture as a quosure
  cat("Expression: ", deparse(get_expr(q)), "\n")
  cat("Environment:", environmentName(get_env(q)), "\n")
  eval_tidy(q)
}

a <- 5
capture(a + 7)
```

If you have ever written a helper like the following and wondered what the curly braces do, here is the answer.

```
my_summary <- function(data, group_col) {
  data |>
    dplyr::group_by({{ group_col }}) |>
    dplyr::summarise(n = dplyr::n())
}
```

`{{ group_col }}` is shorthand for “capture the unevaluated expression the caller passed for `group_col`, and splice it into the `group_by` call”. Without lazy evaluation and quosures this could not work: by the time `group_by` received the argument, R would have already tried to evaluate `group_col` as an ordinary variable, found nothing, and stopped with an error. The whole tidyverse style of naming columns without quotes rests on the promise mechanism in this chapter.

5.5. How targets builds its graph

This is the chapter where `targets` stops being magic. Consider a typical pipeline definition.

```
list(  
  tar_target(raw,    read_csv("data.csv")),  
  tar_target(clean, clean_fn(raw)),  
  tar_target(model, fit(clean))  
)
```

The second argument of each `tar_target()` is a promise. `targets` does not evaluate these promises when you call `tar_target()`. Instead it captures the unevaluated expressions, the same move as `substitute()`, and walks each expression with `codetools::findGlobals()` to discover which other names it mentions. From those references it builds a directed graph. The edges `raw -> clean` and `clean -> model` are inferred at this stage, purely by parsing unevaluated code. No data has been read and no model has been fitted.

Only when you later run `tar_make()` are the promises forced, and even then only in dependency order, and only for targets whose stored hash no longer matches. Every part of this story rests on lazy evaluation. Without it, the pipeline would have to run in order to be planned, which would defeat the entire purpose of planning.

! Key idea: code as data

Lazy evaluation lets R treat a piece of code as *data* before deciding whether, where, or even whether at all to run it. `dplyr` inspects the code to choose where to evaluate it. `targets` inspects the code to build a graph and decide what to skip. This ability to look at code before running it is what people mean when they call R a language well suited to metaprogramming.

5.6. Why this matters for Shiny

A `reactive()` expression is, mechanically, a stored promise that can be forced more than once. The block of code you pass to `reactive()` is captured unevaluated; it is forced only when something downstream asks for its value, and it is re-forced when its inputs have changed. The `isolate()` function is a controlled forcing: it evaluates a reactive's expression without registering a dependency, so the surrounding reactive context does not get told to update when that value later changes. Chapter 7 takes this apart in detail. For now, notice that “a parked computation, evaluated on demand” is the same idea you met as `memoise` in Chapter 3 and will meet again as the reactive graph.

5.7. What you can now predict

- Why an argument that looks like it should error sometimes causes no error, if the function never uses it.

- Why a default argument can be written in terms of another argument.
- Why `dplyr` and friends let you name columns without quotes, and what would break without lazy evaluation.
- Why `targets` can draw and prune a dependency graph before running any of the work.

5.8. Exercises

1. Write `describe_call(x)` that returns a string containing both the unevaluated expression the caller passed and its evaluated value. Use `substitute()`, `deparse()`, and `eval()`. Test it with `describe_call(mean(1:10))`.
2. Build a tiny version of `targets`. Write `graph(...)` that accepts named expressions like `graph(a = 1 + 1, b = a * 2)`, builds a dependency graph by inspecting the unevaluated expressions, and returns the targets in dependency order. Hint: `substitute()` on the `...` and `codetools::findGlobals()` on each expression.
3. Explain, in terms of promises and the environment each promise carries, why this prints 3 and not 1.

```
x <- 1
f <- function(y = x) { x <- 3; y }
f()
```

4. Write a function that takes an argument but never uses it, and pass it `stop("boom")`. Explain why no error occurs. Then add a single line to the function that makes the error appear, and explain precisely what that line did to the promise.

6. The Call Stack

In Chapter 1 we met the stack: a region of memory that grows by a frame when a function is called and shrinks when it returns. R exposes that stack to you. You can read the chain of in-flight calls, inspect the local variables of any frame, and intercept errors and warnings as they travel up it. This chapter is about that machinery and the surprisingly powerful condition system built on top of it.

6.1. Two notions of parent

We have to separate two things the earlier chapters quietly let blur together. Every running function has two relevant environments, and confusing them causes a whole category of advanced R bugs.

The **definition environment**, also called the lexical parent, is where the function was *created*. Closures capture this. It is what governs how free names are resolved, by the rules of Chapter 2. You reach it with `environment(f)`.

The **calling environment**, also called the parent frame, is the environment that contained the expression that *invoked* the function. It is a fact about this particular call, not about the function. You reach it with `parent.frame()`.

```
show_parents <- function() {  
  list(  
    definition = environmentName(environment(show_parents)),  
    calling    = environmentName(parent.frame())  
  )  
}  
  
g <- function() show_parents()  
g()
```

Most ordinary code only ever needs the definition environment, because lexical scoping is what resolves names. The calling environment is the tool you reach for when writing *infrastructure*: code whose job is to inspect or act on its caller. Error handlers, debugging utilities, and the non-standard-evaluation helpers inside `dplyr` all use it. If you are not writing that kind of code, you can mostly forget `parent.frame()` exists, but knowing the distinction is what lets you read other people's infrastructure code.

i Analogy: who wrote me versus who sent me

The definition environment is your birthplace: where you were written, fixed forever. The calling environment is the return address on the envelope of whoever just sent you a task: different every time you are called. Lexical scoping says you look up words in the dictionary of your birthplace, not the dictionary of whoever sent the envelope.

6.2. Inspecting the stack at runtime

When a function calls another, which calls another, R keeps a stack of all the in-flight calls, exactly the stack of frames from Chapter 1. You can read it at any moment.

```
deep <- function() {
  print(sys.calls())      # every call currently on the stack
  print(sys.frames())    # the matching environments
}

a <- function() b()
b <- function() c()
c <- function() deep()

a()
# [[1]] a()
# [[2]] b()
# [[3]] c()
# [[4]] deep()
```

This is the same machinery that powers `traceback()` after an error, `debug()` for stepping into a function, and `browser()` for pausing execution to inspect any frame's local variables. The call stack is not a sealed box. It is a list you can walk, which is why R's debugging tools can be written in R itself.

! Key idea: the stack is data, so the tools are just code

Because the call stack is an ordinary list you can read while a program runs, everything built on top of it is ordinary R reading that list. `traceback`, `browser`, and the entire condition system below are not privileged machinery baked into the language; they are programs that walk the stack. A runtime that lets you see its own bookkeeping is a runtime you can extend in its own terms, which is a theme we will take to its conclusion when we build an evaluator in Chapter 10.

6.3. Conditions: errors, warnings, and messages

R has an unusually well-designed system for signalling and handling exceptional situations, and it is poorly known. It is built on the same stack-walking machinery, and it rewards learning, especially for anyone building pipelines that must not fall over at the first problem.

Two functions carry most of the weight, and the difference between them is the whole point.

tryCatch() catches a condition and *unwinds* the stack: execution leaps out of all the intermediate frames and resumes after the **tryCatch** call. Use it when you want to abandon the failed work and recover.

withCallingHandlers() runs your handler *without* unwinding the stack. The original computation is still sitting there, paused, and can be resumed. Use it when you want a side effect, such as logging, without abandoning the work in progress.

```
safe_fit <- function(model_fn, data) {
  withCallingHandlers(
    model_fn(data),
    warning = function(w) {
      cat("Warning during fit:", conditionMessage(w), "\n")
      invokeRestart("muffleWarning")
    },
    error = function(e) {
      cat("Error during fit:", conditionMessage(e), "\n")
    }
  )
}
```

For pipelines this distinction is not academic. **tar_make()** uses calling handlers internally so that a failure in one target is logged and recorded, and the pipeline carries on with the targets that do not depend on the one that failed. Wholesale unwinding, the **tryCatch** behaviour, would throw away work that had nothing to do with the failure. Choosing the right handler is choosing whether a single failure is local or fatal.

i Analogy: a fire alarm with two settings

tryCatch is a fire alarm that evacuates the whole building: everyone drops what they are doing and leaves by the nearest exit (the stack unwinds). **withCallingHandlers** is an alarm that merely sends a note to the front desk: the note is logged, perhaps an action is taken, and everyone keeps working. Pipelines usually want the second for individual task failures and the first only for genuinely unrecoverable problems.

6.4. Why R has no tail-call optimisation

A note for readers coming from a functional-programming background. R does not optimise tail calls. A function that recurses very deeply will exhaust the C stack and stop with **Error: C stack usage is too close to the limit.**

There are two reasons, and both connect to earlier chapters. First, lazy evaluation (Chapter 4) muddies the very notion of a tail call: a call that looks like it is in tail position may still have to force pending promises after it returns, so the frame cannot simply be discarded. Second, R is built to let you introspect its own stack at any point, as we did above, and quietly folding away frames to save space would break that introspection.

The practical advice is short. Do not use recursion for deep iteration in R. Use `Reduce()`, `purrr::reduce()`, or a plain loop, all of which run in constant stack space. Recursion is still fine for genuinely tree-shaped problems of bounded depth, such as walking a parse tree or a small nested list, where the depth is small and the recursive shape mirrors the data.

6.5. What you can now predict

- Why a function resolves its variable names by where it was written, but can still discover where it was called from when it needs to.
- Why R can offer rich debugging tools written in R itself.
- Why a well-built pipeline can survive a failure in one stage, and which handler makes that possible.
- Why deep recursion blows up in R when it would be fine in some other languages.

6.6. Exercises

1. Write `caller_says()`, a function that, when called from inside another function, deparses and prints the expression the *caller's caller* wrote when invoking the caller. You will need `sys.call(-1)`.
2. Using `withCallingHandlers()`, write `with_logging(f)`, a wrapper that appends every warning emitted by `f` to a log file but lets `f` run to completion normally.
3. Read the help pages for `tryCatch()` and `withCallingHandlers()` alongside this chapter, then write one paragraph, in your own words, on the difference between unwinding and calling handlers, with one example where you would actively prefer each.
4. Write a recursive factorial and call it with a large enough argument to hit the stack limit. Then rewrite it with `Reduce()` and confirm the limit is gone. Explain, using Chapter 1, what changed about the memory the two versions use.

Part III.

Part III: Organising Behaviour

7. Object Systems: S3, S4, R6

R has not one object-oriented system but four. Two of them are functional, dispatching on the class of an argument. One is fully encapsulated and, as you will see, is built directly out of the environments from Chapter 2. This chapter explains each, shows how dispatch actually works as a mechanical lookup, and gives you a rule for choosing.

7.1. S3: the simplest possible object system

An S3 object is any R value with a `class` attribute. There are no class declarations, no inheritance contracts, no validation built in. The whole system is a convention enforced by one primitive, `UseMethod()`, which performs the dispatch.

```
# Constructors: just lists carrying a class attribute.
new_circle <- function(radius) {
  structure(list(radius = radius), class = "circle")
}

new_rectangle <- function(width, height) {
  structure(list(width = width, height = height), class = "rectangle")
}

# A generic function: it promises to dispatch on the class of its argument.
area <- function(shape) UseMethod("area")

# Methods, named generic.class.
area.circle <- function(shape) pi * shape$radius^2
area.rectangle <- function(shape) shape$width * shape$height
area.default <- function(shape) stop("No area method for class ", class(shape))

area(new_circle(5))          # 78.54
area(new_rectangle(3, 4))    # 12
```

The dispatch is entirely mechanical, and worth spelling out because it demystifies a great deal of base R. When you call `area(x)`, R enters the body of `area`, sees `UseMethod("area")`, reads the `class` attribute of `x`, builds the candidate method name `area.circle`, and looks that name up using the ordinary scoping rules from Chapter 2. If the class attribute lists several classes, R tries them in order. If none match, it falls back to `area.default`. There is no registry and no magic; method dispatch is just name construction followed by a normal variable lookup.

7. Object Systems: S3, S4, R6

Most of base R works this way. `print`, `summary`, `plot`, `predict`, `coef`, and `residuals` are all S3 generics. When you fit a linear model with `lm()`, the result is a list with class `"lm"`, and calling `summary(fit)` dispatches to `summary.lm()`. You can write your own `summary.my_class()` for your own objects and it will be found automatically, because dispatch is just a lookup and your method is just a name in scope.

i Analogy: surnames and a phone book

A generic is like calling out a first name, “area!”, in a crowded room. `UseMethod` looks at the object’s class to get a surname, “circle”, and pages the specific person “area.circle” out of the phone book (the scope chain). If nobody by that surname answers, it pages the family default.

7.2. S4: classes that take themselves seriously

S4 is what S3 would look like if the designers had given it formal structure. Classes have named, typed slots. Generics are declared explicitly. Methods are registered with full type signatures, and dispatch can consider more than one argument. The price is verbosity.

```
setClass("Circle",    representation(radius = "numeric"))
setClass("Rectangle", representation(width = "numeric", height = "numeric"))

setGeneric("area", function(shape) standardGeneric("area"))

setMethod("area", "Circle",    function(shape) pi * shape@radius^2)
setMethod("area", "Rectangle", function(shape) shape@width * shape@height)

area(new("Circle", radius = 5))
```

Reach for S4 when you are writing a Bioconductor package, where the entire ecosystem is S4 and conforming is the path of least resistance; when you genuinely need **multiple dispatch**, a method whose behaviour depends on the classes of two or more arguments at once; or when you want slot-level type checking enforced for you. For most everyday work, S3 is enough.

7.3. R6: encapsulated, mutable, and built from environments

R6 is fundamentally different from S3 and S4, and it is where the whole book pays off, because an R6 object *is* an environment. Its methods are functions stored inside that environment, conventionally reached through `self$` and `private$`. And because environments are passed by reference (Chapter 2), R6 objects do not copy when passed to functions. They behave like objects in Python or Java: hand the same object to two functions and both see each other’s changes.

```
library(R6)

Counter <- R6Class("Counter",
```



```

private = list(
  count = 0
),
public = list(
  initialize = function(start = 0) {
    private$count <- start
  },
  increment = function(by = 1) {
    private$count <- private$count + by
    invisible(self)          # returning self enables chaining
  },
  value = function() {
    private$count
  }
)
)

c <- Counter$new(10)
c$increment()$increment(by = 5)
c$value()      # 16

```

Now compare this to `make_counter` from Chapter 3. The two have the same external behaviour: a counter with private, persistent state and methods to change and read it. Under R6, the count lives in the object's private environment. Under the factory pattern, the count lived in the captured environment of the closures. *Both are environments*. The difference is purely the layer of convention R6 adds: explicit `public` and `private` sections, an automatic `self`, an inheritance system, and a constructor. R6 is closures wearing a uniform.

! Key idea: encapsulated objects are environments

S3 and S4 attach behaviour to data through external generics; the data itself is inert. R6 puts the behaviour and the mutable state together inside one environment, and gets reference semantics for free because that is how environments already work. If you understood Chapters 2 and 3, you already understand R6; the rest is naming.

7.4. How to choose

The decision is structural, not a matter of taste. Ask one question: **do I need mutable state shared across method calls?**

If your object is essentially data, a fitted model, a configured plot, a tidy result, use **S3**. Wrap a list in a class, write methods for the generics that matter (`print`, `summary`, `predict`), and stop. This is the right answer the large majority of the time in statistical work.

If you need formal typed slots or multiple dispatch, or you are in the Bioconductor world, use **S4**.

7. Object Systems: S3, S4, R6

If you need an object that remembers things between calls and is naturally about *managing something over time*, a database connection, a streaming file reader, a stateful API client, a cache with explicit invalidation, use **R6**. The encapsulation that S3 lacks is exactly what you want when an object's job is to own the lifecycle of an external resource.

7.5. Why this matters for statistical package design

For a typical statistical or official-statistics package, estimating small-area means, producing indicators, running scenarios, S3 is almost always right. Your objects are estimated quantities: they hold numbers and metadata, not behaviour. Write a constructor that validates its inputs, a compact set of S3 methods so your objects print and summarise and plot sensibly, and resist the urge to reach for a heavier system. This echoes the advice in Hadley Wickham's *Advanced R*: prefer functional design when the domain is mathematical, and the domain of statistics is mathematical.

R6 earns its place when you are managing a *resource* rather than representing a quantity: a session against a statistical office's API that authenticates once and is reused, a reader that must remember its position in a large fixed-width file, a connection pool. There the reference semantics and explicit lifecycle are a feature, not overhead.

7.6. What you can now predict

- Why writing `summary.myclass` is all it takes to make `summary()` work on your objects.
- Why an R6 object handed to two functions lets each see the other's changes, while an S3 list would not.
- Why R6 and the closure-based counter are, underneath, the same mechanism.
- Which object system a new piece of code should use, from a single question about state.

7.7. Exercises

1. Build an S3 class `nuts_estimate` wrapping a regional estimate: a mean, a standard error, a sample size, and a region code. Implement `print`, `summary`, and a `plot` method that draws a horizontal bar with the confidence interval.
2. Convert the `make_counter()` factory from Chapter 3 into an R6 class. Note exactly where you used `<<-` in the closure version and what replaces it in R6, and explain why the replacement works.
3. Read the source of `shiny::reactiveValues`. Identify which object system it uses and explain, in two sentences and with reference to Chapter 2, why that choice is the right one.
4. Take an S3 object you have created and pass it to a function that modifies one of its elements. Confirm the original is unchanged. Now do the same with an R6 object and confirm the original *is* changed. Explain both results using copy-on-modify versus reference semantics.

8. Reactivity

A Shiny app feels alive: change a slider and a plot redraws, and you never wrote the code that connected them. That feeling is produced by the **reactive graph**, a network of dependencies through which invalidation signals flow. It is the R counterpart of the event loop in a browser, and it is built from the closures and lazy evaluation you already understand. Once you can see the graph, Shiny stops being mysterious and starts being predictable.

8.1. The problem reactivity solves

Imagine wiring a small app by hand. Two numeric inputs, one output showing their sum. You could write: when input A changes, recompute the sum; when input B changes, recompute the sum. That works for two inputs and one output.

It does not survive a real application. A real app has dozens of inputs, intermediate computations that depend on each other (cleaning depends on the raw data, the model depends on the cleaning, three tables depend on the model), and outputs that depend on those. Wire it by hand and every time you add or rename an input you must hunt down every handler that touches it and edit them all. You will miss one.

Reactivity inverts the responsibility. Instead of saying “when A changes, recompute the sum”, you *declare* “the sum is whatever A plus B currently is”, and the system works out when and how to recompute it. To do that, it builds a graph of who depends on whom and pushes invalidation signals through it.

i Analogy: a spreadsheet

A spreadsheet is the purest reactive system most people have used. You write `=A1+B1` in a cell. You never write “when A1 changes, update this cell”. The spreadsheet knows the dependency because it read your formula, and it updates the cell whenever A1 or B1 changes. Shiny is a spreadsheet for arbitrary R computations, and the rest of this chapter is how the spreadsheet knows.

8.2. The three kinds of node

Every node in a reactive graph plays one of three roles.

Sources are the roots. They bring values in from outside the reactive system. The familiar one is `input$something`, carrying values from the browser. Others include `reactiveVal()`, which holds a mutable value you set from code, and `reactivePoll()`, which checks an external resource on a timer.

Conductors are interior nodes. A `reactive()` is a conductor: it depends on other nodes and can be depended on in turn. A conductor caches its last computed value and recomputes only when that cache has been invalidated. This is the `memoise` closure from Chapter 3, now with automatic invalidation.

Endpoints are the leaves. They exist for their side effects. The two kinds are `observe()`, which runs purely for effect, and `output$something` assignments, whose effect is to update the browser.

8.3. How a dependency is registered

Here is the part that feels like magic until you see it, and it is built entirely from Chapter 4's promises. When the body of a reactive expression runs and reads `input$x`, two things happen at once. First, the current value of `input$x` is returned. Second, the reactive context that is currently running registers itself in `input$x`'s list of dependents.

From then on, whenever `input$x` changes, it walks its list of dependents and sends each one an **invalidation signal**. The dependents mark themselves invalid but do not recompute immediately. They wait until something actually asks for their value (a conductor) or until the next flush of the event loop (an observer).

```
library(shiny)

ui <- fluidPage(
  numericInput("x", "x:", 1),
  numericInput("y", "y:", 1),
  textOutput("sum")
)

server <- function(input, output) {

  total <- reactive({
    cat("computing total\n")
    input$x + input$y      # reading these registers dependencies on both
  })

  output$sum <- renderText({
    cat("rendering sum\n")
    paste("Sum:", total())  # reading total() registers a dependency on it
  })

  observe({
    cat("observed:", total(), "\n")  # also depends on total
  })
}
```

Draw the graph. `input$x` and `input$y` are sources. `total` is a conductor depending on both. `output$sum` and the `observe()` are endpoints, both depending on `total`. Change `input$x` and

it invalidates `total`, which invalidates both endpoints. The body of `total` runs exactly once, and its cached value is reused by both consumers. That caching is precisely the closure-plus-table pattern from Chapter 3, and the reason `total` runs once rather than twice is the whole reason conductors exist.

! Key idea: reading is subscribing

In a reactive context, reading a value is the same act as subscribing to it. You never write the subscription separately, which is exactly why the code reads like a plain formula. The dependency edge is created as a side effect of evaluating the expression, which is only possible because the expression is a lazily evaluated promise that runs inside a context that is watching what it touches.

8.4. Stopping propagation: `isolate()`

`isolate()` reads a reactive value *without* registering a dependency. It is the equivalent of `event.stopPropagation()` in the browser. Use it to consume a current value without being notified of future changes.

```
observe({
  trigger    <- input$go_button      # depend on the button
  current_x  <- isolate(input$x)     # read x WITHOUT depending on it
  cat("Snapshot of x at button press:", current_x, "\n")
})
```

This is the canonical “take a snapshot when X happens” pattern. The convenience wrappers `eventReactive()` and `observeEvent()` are sugar over exactly this combination of depending on one thing and isolating another.

8.5. The flush cycle

Shiny does not push invalidations one at a time. They accumulate, and at the next **flush** of the session’s event loop, all invalidated reactives are recomputed together, in dependency order. This guarantees that even when several inputs change in the same instant, each downstream node recomputes exactly once, from a consistent set of inputs, and no half-updated intermediate state leaks to the user. It is the same idea as batched re-rendering in modern web frameworks: collect the changes, then settle the whole graph once.

8.6. Two principles for Shiny architecture

Two design rules fall straight out of the graph picture.

Push computation toward conductors. If three outputs each need a cleaned version of the data, do not clean it inside all three. Make one `reactive()` that cleans, and have all three

8. Reactivity

outputs depend on it. Otherwise the cleaning runs three times per change, because each output body evaluates independently. Conductors exist to deduplicate shared work, which is the same benefit memoisation gave you in Chapter 3.

Use endpoints for side effects only. An `observe()` that does heavy computation stalls the flush cycle for the whole session, because the flush is not done until the observer finishes. If you find an observer doing real work, that work almost always belongs in a `reactive()`, with the observer reduced to consuming the result.

8.7. The relationship to targets

A pipeline built with `targets` is, in a real sense, the reactive graph idea applied to long-running computation. Each target is a node. Dependencies are inferred from unevaluated code, exactly the move from Chapter 4. When an upstream input changes, downstream targets are invalidated and recomputed in dependency order. The only real difference is the timescale and where the cache lives. Shiny settles its graph within a session in milliseconds, with the cache in memory. `targets` settles its graph across runs, with the cache on disk, persisting between sessions. Same shape, different clock. We make that explicit in the next chapter.

8.8. What you can now predict

- Why a Shiny output updates without you ever writing the wiring that connects it to its inputs.
- Why a shared computation should live in one `reactive()` rather than being repeated in several outputs.
- Why a heavy `observe()` can make an entire app feel frozen.
- Why `targets` and Shiny feel so similar once you ignore the difference in timescale.

8.9. Exercises

1. Draw the reactive graph for an app with two `numericInputs`, one `reactive()` that adds them, one `renderText` that shows the sum, and one `observeEvent` on a button that snapshots the current sum and prints it. Label every node as source, conductor, or endpoint.
2. In a small app, deliberately introduce the “three consumers, one expensive computation repeated three times” anti-pattern, with `cat()` tracing. Then refactor the computation into a single `reactive()` and confirm from the trace that it now runs once per change instead of three times.
3. Write a tiny reactive system in base R, no Shiny. Provide `make_signal(initial)`, returning something you can get and set, and `watch(signal, callback)`, which runs `callback` whenever the signal is set. Use only closures and environments. You will not reproduce Shiny’s flush cycle, but you will reproduce the read-is-subscribing core.

4. Explain, in terms of dependency registration, why wrapping a value in `isolate()` inside a `reactive()` can stop that reactive from updating when you expected it to. Construct a small example where this is a bug and a small example where it is exactly what you want.

Part IV.

Part IV: Making It Fast

9. Performance

R has a reputation for being slow. The truth is more useful: R is slow when you fight its memory model and fast when you work with it, and the difference can be a factor of thousands on the very same calculation. This chapter cashes in the foundations from Chapter 1. Copy-on-modify is pointers and reference counts. Vectorisation is the cache and the CPU loop. The three layers of caching in the R world are the one idea from Chapter 3 at three timescales.

9.1. Copy-on-modify

R appears to copy a value whenever you assign it to a new name. Physically, it usually does not, and the reason is the pointer picture from Chapter 1. The runtime keeps, for each value, a count of how many names point to it. As long as only one name points to a value, R can modify it in place. The instant a second name points to the same value, the next attempt to modify it through either name forces a real copy first, so the other name is left untouched.

```
library(lobstr)

x <- 1:1e7
obj_addr(x)           # the address of the value on the heap

y <- x                 # no data is copied; y points at the same value
obj_addr(y)           # identical address to x

y[1] <- 99             # NOW the copy happens, to protect x
obj_addr(y)           # a new address
obj_addr(x)           # unchanged
```

This is exactly the locker-and-sticky-note from Chapter 1. `y <- x` copied the note, not the locker. Only when you tried to change the contents through `y` did R duplicate the locker so that `x` would still see the original. The reference count is how R knows whether duplication is necessary: one pointer means safe to edit in place, more than one means copy first.

The sharp practical consequence is the most common performance mistake in R: growing a vector in a loop.

```
# Bad: each iteration may copy the whole vector so far.
x <- c()
for (i in 1:10000) x <- c(x, i^2)      # roughly n-squared work

# Good: allocate once, then fill in place.
```

9. Performance

```
x <- numeric(10000)
for (i in 1:10000) x[i] <- i^2          # linear work

# Best: no R-level loop at all.
x <- (1:10000)^2                      # one operation, run in C
```

The bad version is slow not because R is interpreted but because `c(x, i^2)` keeps creating a new, longer vector and, since `x` still points at the old one, copying everything across each time. Pre-allocating means there is a single value with a single pointer, so each `x[i] <- ...` edits it in place with no copy.

9.2. Why vectorisation is fast

When you write `x + y` on two numeric vectors, R does not run a loop in the interpreter. It calls into compiled C code that walks the two arrays in tight machine instructions, the CPU loop from Chapter 1, and the cost of the R-level interpretation (parsing the expression, dispatching `+`) is paid once for the whole vector rather than once per element.

There is a second, deeper reason, and it is the cache. An R numeric vector is stored as its elements laid out *contiguously* in memory. When the CPU processes it, every expensive trip to RAM pulls in a whole cache line of 64 bytes, eight neighbouring doubles, all of which are about to be used. The hardware's bet from Chapter 1 pays off on every line. A pointer-chasing alternative, where each number sits in a separate object somewhere on the heap, loses that bet on every access and spends most of its time waiting on memory.

So the well-worn advice to “vectorise” is really two wins stacked together: you pay the interpreter's overhead once instead of a million times, and you let the cache work for you instead of against you.

! Key idea: a rule of thumb you can now justify

If you can read a function's body by printing it, it runs in the interpreter and will be slow in a tight loop. If its body is `.Primitive(...)`, `.Internal(...)`, or `.Call(...)`, it runs in C and will be fast over a whole vector. Vectorising means handing the inner loop to C and laying your data out contiguously so the cache cooperates. Both halves come straight from Chapter 1.

```
`+`
# function (e1, e2) .Primitive("+")          run entirely in C

sum
# function (... , na.rm = FALSE) .Primitive("sum")

mean
# function (x, ...) UseMethod("mean")      dispatches, then C
```

9.3. Three layers of memoisation

Caching shows up at three natural scales in the R world, and they are the same idea, a cache plus an invalidation rule, applied at growing scope. You met the engine in Chapter 3; here are its three homes.

Function-call memoisation, via `memoise::memoise` or your eight-line version. The cache is an environment captured in a closure, living in memory. It lasts only for the current R session. Use it for a slow pure function called repeatedly with the same arguments, which is common in Shiny when an expensive result depends on a small set of selectable inputs.

Reactive caching, via `shiny::reactive` and `shiny::bindCache`. The cache lives in the reactive node, and invalidation is automatic through the reactive graph of Chapter 7. It is scoped to a session, or to the whole application with `bindCache(cache = "app")`.

Pipeline memoisation, via `targets`. The cache lives on disk, keyed by a hash of the target's command, its dependencies, and its inputs. It survives across R sessions, across machines if you share the store, and across time. Invalidation is automatic through the pipeline graph.

i One idea, three clocks

In-closure cache, in-session and gone when R exits. Reactive cache, settled within a session in milliseconds. Pipeline cache, on disk and good for months. The art is matching the layer to the lifetime of the work. Caching a one-millisecond function on disk wastes more time on input-output than it saves; trying to use an in-memory cache for a six-hour model fit means recomputing it every time R restarts.

9.4. Concurrency: future and promises

R is single-threaded: one CPU loop, one thing at a time. To run work in parallel across cores or machines, or simply off to the side so a Shiny session stays responsive, use the **future** framework.

```
library(future)
library(promises)

plan(multisession)           # run futures in background R processes

f <- future({
  Sys.sleep(3)
  "done"
})

# The current session is free to keep working here.
value(f)                     # blocks only when you finally need the result
```

9. Performance

Inside Shiny, the **promises** package provides a then-style operator, `%...>%`, so an output can be declared as “render the result of this future when it finishes, without blocking the session in the meantime”.

```
output$plot <- renderPlot({  
  future({ slow_computation(input$x) }) %...>% plot()  
})
```

A word of warning about vocabulary. The word *promise* here is not the same as the promise from Chapter 4. There it meant the unevaluated-expression mechanism behind lazy evaluation. Here it means a value that will be ready later, the concurrency sense familiar from JavaScript. R reuses the word for two different ideas. Context disambiguates, but the collision is real, so name your variables with care.

9.5. Three rules for Shiny performance

Everything above condenses into three operational rules.

Pre-allocate. Never grow a vector inside a reactive block. If you are, you have almost certainly missed a vectorised primitive that would do the whole thing at once.

Push heavy CPU off the main thread. Anything taking more than about a tenth of a second belongs in a **future**, so the session stays responsive. Fill the visual gap with a spinner from **shinycssloaders**.

Cache at the right layer. Per-session transient results go in reactivities. Results shared across sessions go in `bindCache(cache = "app")`. Results that must survive a restart go in **memoise** with a disk backend, or as a **targets** stage if they are part of a pipeline.

9.6. What you can now predict

- Why `y <- x` is instant even for a huge `x`, and exactly when the hidden copy finally happens.
- Why growing a vector in a loop is quadratic, and why pre-allocation fixes it.
- Why a vectorised operation beats an R loop twice over, in interpreter overhead and in cache behaviour.
- Which of the three caching layers a given computation belongs in, from how long its result needs to live.

9.7. Exercises

1. Use `lobstr::obj_addr` and `tracemem()` to demonstrate copy-on-modify in three cases: assigning a vector to a new name, then modifying one element of the new vector, then modifying an element via `data.table`'s `:=` operator instead of base assignment. Explain each address change, or lack of one, using reference counts.

2. Time the three vector-building approaches (`c()` in a loop, pre-allocate then fill, and the fully vectorised form) for a vector of length one million. Plot the three timings and explain the ranking from this chapter.
3. Take a genuinely slow pure function, a bootstrap with many resamples is ideal, and wrap it in `memoise::memoise()` with a filesystem backend. Re-run it from a fresh R session and confirm the second call is instant. Which of the three caching layers did you just use, and why was a disk backend necessary?
4. In a small Shiny app, make a slow plot render asynchronously with `future` and `promises`, and confirm the rest of the app stays responsive while it computes. Then explain, using the single-threaded model, what would have happened without the future.

Part V.

Part V: Code as Data

10. Metaprogramming: Code as Data

In Chapter 4 we saw something quietly radical. When you write `filter(df, x > 3)`, R does not run `x > 3` and pass the answer. It hands the function the *unevaluated code* and lets the function decide what to do with it. That was our first glimpse of an idea that runs all the way through R: **code is data**. A piece of R code is an ordinary object that you can store in a variable, take apart, build up piece by piece, and choose to run later, or not at all.

This chapter makes that idea explicit and hands you the small toolkit R provides for working with code as data. We will take it slowly, one tool at a time, because the next chapter uses these tools to do something that sounds impossible and turns out to be short: build a working interpreter for a small language, in R itself.

10.1. Capturing code without running it

The most basic move is to grab a piece of code and hold it, unevaluated. The tool is `quote()`.

```
e <- quote(1 + 2)
e
#> 1 + 2
```

Nothing was added. `e` is not 3. It is the *expression* `1 + 2`, sitting in a variable, waiting. You can ask what kind of object it is, and you can run it later with `eval()`.

```
class(e)
#> [1] "call"

eval(e)
#> [1] 3
```

So `quote()` captures, and `eval()` runs. Between those two moments, the code is just an object you can examine and change like any other. That gap, between capturing code and running it, is the entire playground of metaprogramming.

i Analogy: a recipe card versus a cooked meal

`quote(1 + 2)` is a recipe card that reads “add 1 and 2”. It is not the meal. You can read the card, copy it, scribble on it, file it away, or hand it to someone else. `eval()` is the act of actually cooking from the card. Most programming hands you meals; metaprogramming lets you handle the cards.

10.2. Code is a tree

Once you are holding a piece of code, you can look inside it, because an R expression has a simple, regular structure: it is a *tree*. The technical name is an abstract syntax tree, but the idea is plain. Take this expression:

```
e <- quote(f(x, y + 1))
```

A call like this behaves like a list. Its first element is the function being called, and the rest are the arguments.

```
e[[1]]      #> f          (the function)
e[[2]]      #> x          (first argument)
e[[3]]      #> y + 1      (second argument, itself a smaller call)

as.list(e)
#> [[1]] f
#> [[2]] x
#> [[3]] y + 1
```

Notice that the second argument, `y + 1`, is itself a call with its own pieces. Expressions nest inside expressions, which is exactly what makes them a tree.

Now the reveal that ties this to the rest of the book. **In R, everything that happens is a function call**, including the things that do not look like function calls. Arithmetic is a function call. So is an `if`, an index with `[]`, even a block of code in curly braces. You can write them in their hidden call form:

```
`+`(1, 2)      #> 3
`if`(TRUE, "yes", "no") #> "yes"
`[`(c(10, 20, 30), 2)  #> 20
```

The backticks just let you refer to these operators by their real names, which are ordinary functions. This uniformity is a gift inherited from Lisp: every expression is one of only three things. It is a **constant** (like `1` or `"hello"`), a **name** (like `x`, a reference to a variable), or a **call** (a function applied to some argument expressions). That is the whole grammar. Because the grammar is so small and regular, code is genuinely easy to treat as data, and in the next chapter we will lean on exactly these three cases.

10.3. Walking and rewriting code

Because a call is list-like, you can not only read its parts but change them.

```
e <- quote(a + b)
e[[1]] <- quote(`*`)  # replace the operator
e
#> a * b
```

You can also walk an entire expression tree and compute something about it. Here is a small function that counts how many additions appear anywhere in an expression. It calls itself on each piece, which is safe here because expression trees are shallow (recall from Chapter 5 that bounded-depth recursion is fine in R; it is only deep iteration you must avoid).

```
count_adds <- function(expr) {
  if (is.call(expr)) {
    here <- if (identical(expr[[1]], quote(`+`))) 1 else 0
    here + sum(vapply(as.list(expr)[-1], count_adds, numeric(1)))
  } else {
    0 # a constant or a name contains no additions
  }
}

count_adds(quote((a + b) * (c + d + e)))
#> [1] 3
```

Read what just happened: we wrote a program that takes another program as input and answers a question about it. That is metaprogramming in one breath. Tools like linters, code formatters, and coverage trackers are elaborate versions of `count_adds`.

10.4. Building code from scratch

Just as you can take code apart, you can put it together. The base tool is `bquote()`, which quotes an expression but lets you splice values in using `.`.

```
v <- 5
bquote(x + .(v))
#> x + 5
```

The `x` stayed as code, but `.(v)` was replaced by the current value of `v`. You can also assemble a call directly from its parts:

```
call("sum", 1, 2, 3)
#> sum(1, 2, 3)
```

This matters because it lets a program generate code tailored to its inputs: a function that, given a column name, builds the right summary expression, or a report generator that assembles analysis code from a configuration.

10.5. Running code needs an environment

We met `eval()` above, but there is a second ingredient we glossed over. To *run* a piece of code, you need two things: the code itself, and a place to look up the names in it. That place is an environment, exactly the side-table from Chapter 2.

```
e <- quote(x + 1)

eval(e)
#> Error: object 'x' not found      (no 'x' in scope here)

eval(e, list2env(list(x = 10)))
#> [1] 11                          (now 'x' is 10, in the environment we gave)
```

The same expression gives an error or the answer 11, depending only on the environment you evaluate it in. This is the deepest point of the chapter, and it is worth stating plainly.

! Key idea: running code is an expression plus an environment

Code on its own is inert. To run it you must pair it with an environment that gives meaning to its names. Expression plus environment is the irreducible recipe for evaluation. Hold this pair in mind: it is literally the two arguments to `eval()`, and in the next chapter it becomes the two arguments to the evaluator we build ourselves.

10.6. Quasiquote: the modern toolkit

The `rlang` package, which powers the tidyverse, wraps these ideas in a tidier interface called **quasiquote**. You capture code with `expr()` and splice values or other code back in with `!!` (pronounced “bang-bang”).

```
library(rlang)

v <- 5
expr(x + !!v)
#> x + 5
```

This is the same move as `bquote's` `.()`, just with friendlier names. And it closes a loop from Chapter 4. When you write a tidyverse helper with `{ col }`, you are using a shortcut for “capture the caller’s unevaluated expression with `enquo`, then splice it back in with `!!`”. Now you can see all the way down: `{ }` is metaprogramming with a comfortable handle on it.

10.7. A statistician’s daily metaprogramming: formulas

You already use code as data, probably without naming it, every time you fit a model. A formula like `y ~ x + z` is not computed when you write it. It is a captured, unevaluated expression that also remembers the environment it was written in.

```
f <- y ~ x + z
class(f)
#> [1] "formula"
```

```
f[[1]]      #> ~ ~
f[[2]]      #> y
f[[3]]      #> x + z
```

A formula is a tree, just like the expressions above, with `~` as its operator. This is why `lm(y ~ x, data = d)` can look up `y` and `x` as columns of `d`: the formula handed `lm` the unevaluated code, and `lm` chose where to evaluate it, exactly as `filter` did in Chapter 4. Every modelling function in R is doing metaprogramming on your behalf.

10.8. The boundary is thin: data made of functions

Here is a small demonstration that is worth a few minutes of wonder, because it shows the code-and-data duality from the other side. We have seen that code can be treated as data. It is also true that data can be built out of nothing but functions. Suppose R had no way to pair two values together. We could *invent* the pair using only closures from Chapter 3.

```
cons <- function(a, b) function(selector) selector(a, b)
car  <- function(pair) pair(function(a, b) a)
cdr  <- function(pair) pair(function(a, b) b)

p <- cons(1, 2)
car(p)  #> 1
cdr(p)  #> 2
```

There is no list and no vector here. A “pair” is a closure that has captured two values and waits to be told which one to hand back. `car` and `cdr` ask for the first or the second. You would never build pairs this way in real R, but the exercise reveals something deep: the line between a piece of data and a piece of code is much thinner than it looks. The next chapter walks straight across that line.

Why this matters for your work

Once you can see metaprogramming, the tidyverse stops being a collection of special cases. `dplyr` verbs, `ggplot2`’s `aes()`, `data.table`’s `j` argument, model formulas, and the way `targets` reads your pipeline (Chapter 4) are all the same move: capture code, inspect or transform it, and evaluate it in a chosen environment. Understanding the move lets you read these libraries’ source with comprehension, debug their surprising error messages, and write your own small tools that take code as input.

10.9. What you can now predict

- Why the same expression can error in one place and succeed in another, with nothing changed but the environment.

10. Metaprogramming: Code as Data

- Why `lm(y ~ x, data = d)` can find `y` and `x` in a data frame even though they are not variables in your console.
- Why the tidyverse can let you name columns without quotes, and what `{ }` is shorthand for.
- How a tool could read your code and answer questions about it without running it.

10.10. Exercises

1. Use `quote()` to capture `mean(x) + sd(y)`. Without running it, extract just the two function names being called. Then evaluate the whole expression in an environment where `x` and `y` are vectors you supply.
2. Extend `count_adds` into `count_calls(expr, fname)` that counts how many times any given function is called inside an expression. Test it by counting the `*` operations in `quote(a * b + c * d * e)`.
3. Write a function `make_power(n)` that *builds and returns the code* for raising its argument to the power `n`, using `bquote`. For example `make_power(3)` should return the expression `x ^ 3`. Then evaluate it for a specific `x`.
4. Confirm that a formula carries its environment. Create `y ~ x` inside a function where a local variable `secret` exists, return the formula, and from outside the function recover `secret` through the formula's environment. Explain the result using Chapter 2.

11. The Metacircular Evaluator

This is the chapter the whole book has been walking toward. Its title sounds forbidding, so let us defuse it immediately. A **metacircular evaluator** is a program that runs a language, written *in that same language*. We are going to write a small interpreter for a useful slice of R, in R. When it works, you will have built, with your own hands, a tiny version of the very thing that has been running your code all along.

The reason this is the climax of a book called *Think Like the R Runtime* is simple. There is no surer way to understand a runtime than to build one. Every idea from the earlier chapters will reappear here, not as a topic but as a line of code we cannot do without: environments become the place we look up names, closures become the way our functions remember where they were born, and the expression-plus-environment recipe from Chapter 9 becomes the signature of our main function.

We will build it in small, safe steps, testing after each one. Do not skim. Type each piece into an R session and watch it work, because the understanding is in the watching.

11.1. The plan: eval and apply

Running code turns out to be a conversation between two functions that call each other.

To **evaluate** an expression, you look at what kind of expression it is. If it is a number, it is its own value. If it is a name, you look it up. If it is a call, you evaluate the function and each of its arguments, and then you **apply** the function to those argument values.

To **apply** a function to some arguments, you create a fresh environment, bind the arguments to the function's parameter names in it, and then **evaluate** the function's body in that new environment.

Read those two paragraphs again and notice that each mentions the other. Evaluate calls apply; apply calls evaluate. This mutual recursion is the engine of every interpreter ever written. Everything else is detail.

We need just two helpers first, for our environments. We will not reinvent them: we will use R's own environments from Chapter 2, which already do everything we need, including the parent chain.

```
env_new <- function(parent) new.env(parent = parent)
env_get <- function(env, name) get(name, envir = env, inherits = TRUE)
env_set <- function(env, name, value) assign(name, value, envir = env)
```

`env_get` uses `inherits = TRUE`, which means: if the name is not in this environment, walk the parent chain and keep looking. That one flag is lexical scoping, handed to us for free.

11.2. Step 1: the simplest expressions

We begin `lang_eval`, our evaluator. Its two arguments are an expression and an environment, exactly the pair Chapter 9 told us we would need. The two easiest cases are constants and names.

```
lang_eval <- function(expr, env) {
  # A constant evaluates to itself.
  if (is.numeric(expr) || is.character(expr) || is.logical(expr)) {
    return(expr)
  }
  # A name is a variable: look it up in the environment.
  if (is.symbol(expr)) {
    return(env_get(env, as.character(expr)))
  }
  stop("lang_eval: I do not yet understand: ", deparse(expr))
}
```

We can already evaluate the two simplest kinds of code. Note that we use `quote()` from Chapter 9 to hand our evaluator a piece of unevaluated code.

```
e <- env_new(emptyenv())
env_set(e, "x", 10)

lang_eval(quote(42), e)    #> 42      a constant is itself
lang_eval(quote(x), e)    #> 10      a name is looked up
lang_eval(quote(y), e)    #> Error: object 'y' not found
```

The constant returns itself. The name `x` is found in the environment. The name `y` is not, and the error comes from `env_get` walking the (empty) parent chain and finding nothing, precisely the lookup mechanism of Chapter 2.

11.3. Step 2: function calls

Now the interesting case: a call, such as `1 + 2`. Recall from Chapter 9 that a call is a tree whose first element is the function and whose remaining elements are the argument expressions. To evaluate a call, we evaluate the function, evaluate each argument, and apply one to the others. For now our “functions” will be ordinary R primitives like `+`, which we can apply with `do.call`.

```
lang_eval <- function(expr, env) {
  if (is.numeric(expr) || is.character(expr) || is.logical(expr)) {
    return(expr)
  }
  if (is.symbol(expr)) {
    return(env_get(env, as.character(expr)))
  }
}
```

```

if (is.call(expr)) {
  fn   <- lang_eval(expr[[1]], env)           # evaluate the function
  args <- lapply(as.list(expr)[-1], lang_eval, env = env) # evaluate args
  return(do.call(fn, args))                   # apply
}
stop("lang_eval: I do not understand: ", deparse(expr))
}

```

To make this run we need an environment in which the names `+`, `*`, and so on refer to the real arithmetic functions. This is our language's global environment.

```

global_env <- env_new(emptyenv())
env_set(global_env, "+", `+`)
env_set(global_env, "*", `*`)
env_set(global_env, "-", `-`)
env_set(global_env, "<", `<`)

lang_eval(quote(1 + 2), global_env)      #> 3
lang_eval(quote(2 * (3 + 4)), global_env) #> 14

```

Trace the second one. The outer call is `*`. We evaluate its function: the name `*` is looked up in `global_env` and returns R's multiplication function. We evaluate its two arguments: `2` is a constant, and `3 + 4` is itself a call, so `lang_eval` recurses, evaluates `+`, `3`, and `4`, and applies, giving `7`. Then we apply `*` to `2` and `7`, giving `14`. Our evaluator is walking the tree exactly as Chapter 9 described, and the mutual recursion between evaluating and applying is already visible: a call's arguments are evaluated by the very same function.

11.4. Step 3: a special form, *if*

Not everything can be handled by “evaluate all the arguments, then apply”. Consider `if`. If we evaluated both branches before choosing, an `if` guarding against an error would run the error anyway. So `if` must be handled specially: evaluate the condition, then evaluate only the chosen branch. Constructs that bend the normal evaluate-everything rule are called **special forms**, and every language has a handful.

We add a check for `if` before the general call case.

```

lang_eval <- function(expr, env) {
  if (is.numeric(expr) || is.character(expr) || is.logical(expr)) {
    return(expr)
  }
  if (is.symbol(expr)) {
    return(env_get(env, as.character(expr)))
  }
  if (is.call(expr)) {
    op <- expr[[1]]

```

```

# Special form: if
if (identical(op, quote(`if`))) {
  if (isTRUE(lang_eval(expr[[2]], env))) {
    return(lang_eval(expr[[3]], env)) # the "then" branch
  } else {
    return(lang_eval(expr[[4]], env)) # the "else" branch
  }
}

# General case: ordinary function application
fn  <- lang_eval(op, env)
args <- lapply(as.list(expr)[-1], lang_eval, env = env)
return(do.call(fn, args))
}
stop("lang_eval: I do not understand: ", deparse(expr))
}

```

```

lang_eval(quote(if (1 < 2) 10 else 20), global_env) #> 10
lang_eval(quote(if (5 < 2) 10 else 20), global_env) #> 20

```

11.5. Step 4: functions of our own, with closures

So far our language can only use functions that already exist. The final step, and the one that makes this a real language, is letting our language *define* its own functions. This is where every idea from Part II comes together at once.

When our evaluator meets a **function** expression, it must build a closure. From Chapter 3 we know exactly what a closure is: the parameters, the body, and a pointer to the environment in which it was defined. We capture all three.

When our evaluator later *applies* one of these closures, it must, from Chapters 2 and 3: create a new environment whose parent is the closure's definition environment (this is what makes scoping lexical), bind the arguments to the parameter names in that new environment, and evaluate the body there.

Here is the finished evaluator, with the **function** special form and a proper `lang_apply` that handles both R primitives and our own closures.

```

lang_eval <- function(expr, env) {
  if (is.numeric(expr) || is.character(expr) || is.logical(expr)) {
    return(expr)
  }
  if (is.symbol(expr)) {
    return(env_get(env, as.character(expr)))
  }
  if (is.call(expr)) {
    op <- expr[[1]]

```

```

if (identical(op, quote(`if`))) {
  if (isTRUE(lang_eval(expr[[2]], env))) {
    return(lang_eval(expr[[3]], env))
  } else {
    return(lang_eval(expr[[4]], env))
  }
}

# Special form: function -> build a closure that remembers its birthplace
if (identical(op, quote(`function`))) {
  return(list(
    kind    = "closure",
    params  = names(expr[[2]]), # the parameter names
    body    = expr[[3]],        # the unevaluated body
    env     = env               # the definition environment
  ))
}

# General case: evaluate the function and the arguments, then apply
fn  <- lang_eval(op, env)
args <- lapply(as.list(expr)[-1], lang_eval, env = env)
return(lang_apply(fn, args))
}
stop("lang_eval: I do not understand: ", deparse(expr))
}

lang_apply <- function(fn, args) {
  # A built-in R function (like + or *).
  if (is.function(fn)) {
    return(do.call(fn, args))
  }
  # One of our own closures.
  if (is.list(fn) && identical(fn$kind, "closure")) {
    call_env <- env_new(parent = fn$env) # new frame, parented at birthplace
    for (i in seq_along(fn$params)) {
      env_set(call_env, fn$params[[i]], args[[i]]) # bind arguments
    }
    return(lang_eval(fn$body, call_env)) # run the body there
  }
  stop("lang_apply: not something I can call")
}

```

Now the moment of truth. We will build, in our own language, the `make_adder` factory from Chapter 3, and check that it gets closures right.

```
add_maker <- lang_eval(quote(function(n) function(x) x + n), global_env)

add5 <- lang_apply(add_maker, list(5))
add9 <- lang_apply(add_maker, list(9))

lang_apply(add5, list(10))    #> 15
lang_apply(add9, list(10))    #> 19
```

Stop and savour this. Our hand-written evaluator just produced two functions that share a body and behave differently, because each remembers a different `n`. We did not build special machinery for that. It fell out of one line in `lang_apply`, the line that parents the new environment at `fn$env`. Lexical scoping and closures are not features we added; they are consequences of where we pointed a parent pointer, exactly as the book has claimed since Chapter 2.

! Key idea: the runtime is not magic, because you just built one

Every behaviour we have described across nine chapters, name lookup walking a parent chain, closures remembering their birthplace, the evaluate-then-apply rhythm of running a call, is now sitting in front of you as a few dozen lines of ordinary R. The interpreter that runs your real R code is vastly larger and faster, but it is the same shape. You are no longer guessing what the runtime does. You have written its essence.

11.6. Why it is called metacircular

Our evaluator is written in R, and it evaluates R. The language doing the interpreting and the language being interpreted are the same. That circularity is the point of the name, and it is also a genuine intellectual loophole worth being honest about: we did not have to explain how `+` adds, or how `do.call` calls, because we borrowed those directly from the R underneath us. A metacircular evaluator explains a language *in terms of itself*, which is illuminating about structure (how scoping, closures, and special forms fit together) while quietly leaning on the host language for the very bottom. To explain that bottom you would descend to Chapter 1: the C code, the registers, the memory. The two ends of this book meet here.

11.7. The real thing

You have been using a more powerful version of `lang_eval` and `lang_apply` all along. R exposes them directly. `eval()` is the real evaluator, and `quote()` (Chapter 9) is how you hand it code, and an environment is where it runs.

```
eval(quote(2 * (3 + 4)))    #> 14
```

When you call `eval`, R does, in fast and battle-tested C, what our `lang_eval` did in R: it dispatches on the kind of expression, looks names up by walking environments, treats `if` and `function` and a few others as special forms, and applies functions by making a new evaluation

environment parented at each function’s definition environment. You now know what is inside that call, because you wrote a small one yourself.

11.8. What you can now predict

- Why interpreting a language is a conversation between an “evaluate” step and an “apply” step.
- Why some constructs (`if`, `function`, and in R `quote` itself) cannot be ordinary functions and must be special forms.
- Why closures and lexical scoping require nothing more than parenting a call’s environment at the function’s definition environment.
- What R’s own `eval` is doing, in outline, every time it runs a line of your code.

11.9. Exercises

1. Add multiplication-free subtraction’s friend: extend `global_env` with a new primitive, for example `%%` for remainder, and confirm your evaluator can use it with no change to `lang_eval`. Why did no change to the evaluator’s code turn out to be necessary?
2. Add a `let` special form so that `let(x, 5, x + x)` evaluates the third argument in a new environment where `x` is bound to 5. Where in `lang_eval` does it belong, and why must it be a special form rather than an ordinary function?
3. Our evaluator is eager: it evaluates every argument before applying. Make one argument lazy. Change `lang_apply` so that instead of receiving evaluated arguments it receives the unevaluated argument expressions plus the calling environment, and evaluates each only when first needed. You will have reinvented the promise from Chapter 4.
4. Add an error if a closure is called with the wrong number of arguments. Then explain, in terms of `lang_apply`, why R’s own “unused argument” and “missing argument” errors exist and where in an interpreter they must be detected.

12. Capstone: One System, Every Idea

Every idea in this book exists, in some form, inside any non-trivial R system. To show that they are not ten separate topics but one connected way of thinking, this final chapter walks through a single realistic system and points to where each idea lives. We will not build the whole thing line by line; we will trace the machinery.

12.1. The scenario

Picture a small application built for an agricultural statistics office. It reads administrative data on holdings and yields, fits a small-area estimate of mean output for each region, caches the fits, and serves the results through a Shiny dashboard that lets a user filter regions, drill into individual estimates, and download the underlying data. It is the kind of system a working statistician actually ships, and it touches every chapter.

12.2. Where each idea lives

12.2.1. Foundations (Chapter 1)

The data is large, so memory layout is not an afterthought. The yield series are held as plain numeric vectors, contiguous in memory, so that the per-region summaries run as vectorised operations that the cache cooperates with rather than as pointer-chasing loops. The decision to keep the heavy reference tables in one shared structure, loaded once, rather than rebuilt per request, is a direct application of “moving data is the expensive part”.

12.2.2. Environments (Chapter 2)

Helper functions live in an `R/` directory and are sourced into a clean environment by `_targets.R`. The pipeline’s definition environment therefore contains only those helpers, with no scratch objects and no test data, so nothing irrelevant is dragged into a target’s dependency footprint. Scope discipline here is a performance and correctness decision, not a stylistic one.

12.2.3. Closures (Chapter 3)

The fitting routine is a function factory. `make_estimator(prior, model_spec)` returns a function whose only argument is the data, with the prior and the specification captured in its environment. Swapping priors or specifications is then a one-line change, and the data-flow code never has to mention configuration at all.

12.2.4. Lazy evaluation (Chapter 4)

The pipeline is declared, not run. `tar_target(estimate, fit_per_region(clean, model_spec))` captures an unevaluated expression. `targets` reads that expression, discovers that `estimate` depends on `clean` and `model_spec`, and adds the edges to its graph, all before any fitting happens. The plan exists because the code could be inspected as data.

12.2.5. The call stack and conditions (Chapter 5)

Each per-region fit is wrapped in `withCallingHandlers()`. A region that fails to converge is logged with its diagnostic and replaced by a sentinel value, while the remaining regions continue. The pipeline finishes successfully and carries a record of which regions failed and why. A single bad region is a local event, not a fatal one, because we chose a calling handler over an unwinding one.

12.2.6. Object systems (Chapter 6)

Each estimate is an S3 object: a list with class `nuts_estimate`, holding the posterior summary, the relevant data subset, and metadata. `print` gives a one-line view, `summary` the full posterior table, `plot` a forest plot of regional estimates with intervals. R6 is reserved for the one component that manages a resource over time: the connector that authenticates once against the source database and is reused across the run.

12.2.7. Reactivity (Chapter 7)

The dashboard loads the pipeline’s output once when a session starts. A single `reactive()` filters the estimates to the regions the user has selected. Three downstream outputs, a table, a map, and a forest plot, all depend on that one conductor, so the filter runs once per change rather than three times. The graph deduplicates the shared work automatically.

12.2.8. Performance (Chapter 8)

The pipeline is memoised on disk by `targets`: only regions whose underlying data changed are refitted when the pipeline is rerun. Inside the dashboard, the map render is slow because it depends on a large shapefile, so it runs inside a `future` and the session stays responsive while it draws. The shapefile is loaded once at startup, not per session, the same “load shared data once” decision as in the foundations.

12.2.9. Metaprogramming (Chapter 9)

Code-as-data is not only inside `targets`; it surfaces in the analysis itself. The models are specified with formulas, `output ~ region + year`, which are captured, unevaluated expressions that the fitting code evaluates against each region’s data. A small internal helper builds those formulas programmatically from a configuration file, so that adding a covariate is a change to data, not to code. When the dashboard lets an analyst type a custom filter, that text is parsed into an

expression, checked by walking its tree to forbid anything dangerous, and only then evaluated in a controlled environment. Every one of these is the capture-inspect-evaluate move from Chapter 9.

12.2.10. Building a runtime (Chapter 10)

The evaluator chapter does not ship inside this system, and it is not meant to. Its role is what it leaves behind in the person who built it. The engineer who has written `lang_eval` reads the `targets` source and recognises the dependency walk as an expression traversal. They read a confusing tidyverse error and locate it as a failure in the capture-and-evaluate step. They reason about a Shiny reactive's scope without hesitation, because they have parented an environment at a definition environment with their own hands. The interpreter is the rung at the top of the ladder: having built one, the reader sees every other piece of machinery in this system as a variation on something they understand from the inside.

12.3. What you have learned

Read that list again and notice that the ideas are not really separable. The factory in Chapter 3 only works because of the lexical scoping in Chapter 2, which only makes sense given the heap in Chapter 1. The pipeline graph in Chapter 4 is invalidated by the same kind of machinery as the reactive graph in Chapter 7. The on-disk cache in Chapter 8 is the in-closure cache from Chapter 3, grown up and moved to disk. The reference semantics that make R6 useful in Chapter 6 are the same reference semantics that make environments the one exception to copy-on-modify in Chapter 2. The metaprogramming of Chapter 9 is the lazy evaluation of Chapter 4 made deliberate, and the evaluator of Chapter 10 is all of it at once, assembled into a small machine that you built. It is one fabric.

The argument that opened this book, borrowed from the course that inspired it, was that you cannot reason about a language by memorising rules: you have to be able to predict what the runtime will do. Everything here has been in service of that, for R. You now have a working model that runs from electrical switches up to the reactive graph, and a habit to go with it.

The habit

When R surprises you, do not look up the rule. Predict, from the model, what should happen, then run a tiny experiment to check. When the prediction and the experiment disagree, you have found a gap in the model, and closing it is how the model gets sharper. Read the source of a function you have always taken for granted. Print an object you have always just used. Ask why, and answer with mechanism. That habit, more than any single fact in this book, is what makes the difference.

12.4. Where to go next

Three books go deeper than this course, and you are now ready for them. Hadley Wickham's *Advanced R* covers environments, functional programming, and metaprogramming in full depth,

12. Capstone: One System, Every Idea

and is the natural next step. Norman Matloff's *The Art of R Programming* is excellent on performance and debugging from a systems perspective. For Shiny in production, *Engineering Production-Grade Shiny Apps* by Colin Fay and colleagues builds on the reactive model from Chapter 7. For pipelines, the `targets` manual by Will Landau is thorough and free online.

Beyond reading, the best next step is to take a piece of R you rely on, a package you use, a Shiny app you maintain, a pipeline you run, and explain it to yourself using the model in this book. Find the environments. Find the closures. Find the graph. Find the cache. When you can do that for code you did not write, the model is yours.